Kampung Sambau - Of the Dark

Team: Siskamling | Hack Apertus, Track 2B - Own Project
Technical report | 15 October 2026
Repository: https://github.com/ratafani/Scary-Gate-Keeper
Public demo: https://scary-gate-keeper.vercel.app/

1. Purpose and contribution

Kampung Sambau is a browser-based 3D social-deduction game. The player guards a village at night, interviews visitors, inspects their ID cards and resident information, and decides whether to admit them. Apertus gives each fictional visitor a character-specific voice and translates natural-language requests into visible actions. This is an entertainment prototype, not real-world identity verification.

The new hackathon work includes the React/Three.js web implementation, Apertus integration, structured action validation, Docker packaging, shared session persistence, responsive illustrated interface, and conversion of authored walk, jump and turn animations. The existing, team-owned NeighborhoodWatchVision concept, village, character assets and scenario approach were reused. This provenance is disclosed against the requirement that the submitted project be new; organizer interpretation of reused assets remains applicable.

2. Purposeful use of Apertus

The runtime model is swiss-ai/Apertus-v1.5-70B through the configurable OpenAI-compatible CSCS endpoint. No other runtime language model is used. The system prompt supplies the selected English/German language, private character profile, ID facts, behavioral constraints and recent conversation. Apertus must return exactly a speech string and an action enum: none, turn_around or jump.

The model decides whether a request requires an action. The application validates the complete JSON envelope with Zod and permits one Apertus repair attempt for invalid output. Commands are not inferred from player keywords or dialogue prose. An accepted response carries unique action and encounter IDs so repeated requests work and delayed replies cannot affect a different visitor. Trusted server logic determines scoring and anomaly jump descent; model output cannot set scores or physics parameters.

Requests use temperature 0.35, max_tokens 240, JSON-object response mode, and a 45-second timeout per completion. A repair may require a second completion. Recent dialogue is bounded to 12 messages. This keeps prompts bounded but is not a measured cost or scalability guarantee.

3. Architecture and data

The browser renders bundled village and character GLBs using React, TypeScript, Three.js and WebGL2. A Node/Express service owns private profiles, scoring and the Apertus credentials. Chat and gate decisions use JSON HTTP endpoints. A shift chooses eight scenario entries without replacement from 13 authored encounters; recurring character identities may have different scenarios. Five resident profiles support investigation.

Visitors arrive and leave using the authored Walk clip. Rejecting sends the NPC right outside the closed gate; admitting opens the gate and sends the NPC left into the village. Turn uses authored stepping and a four-second 360-degree body rotation. Jump uses authored poses; anomaly descent is stretched relative to ordinary descent. The supplied Alfa animation rig is retargeted onto the shared web visitor geometry. Sampled morph animation preserves materials but increases download size. Fatih/Handaru-specific FBX geometry is not separately imported.

The Docker deployment stores sessions in process memory with a two-hour expiry and a bounded local session count. The public Vercel demo uses Upstash Redis because function instances do not share memory. Redis stores session JSON and conversation history with expiry. Atomic compare-and-set updates reject stale

concurrent writes. Redis credentials and the inference key remain server-side. The data directory is 25,378 bytes, below the 100 MB limit.

The microphone path records WAV and requests transcription from the same configured inference endpoint. Audio capability and Swiss German recognition quality are unverified; text input is the supported evaluation path. No microphone accuracy claim is made. Recordings are not persisted by the app. In the public demo, text/history is processed by Vercel, Redis and the inference provider; provider retention is outside the application's guarantees.

4. Mandatory deployment architecture

Selected target: on-premise, under the deploying organization's administration. The root make run command launches the Node application through Docker Compose, serves bundled assets locally and binds port 3000 to host loopback. It requires Docker Compose v2, a WebGL2 browser and a reachable Apertus-compatible inference service. LLM_NAME, LLM_BASE_URL and LLM_API_KEY configure that service.

For an organization-controlled deployment, run the application container and Apertus inference endpoint on infrastructure administered by that organization, and point LLM_BASE_URL to its endpoint. Keep sessions in the application's built-in memory store for a single instance; no Vercel or Upstash dependency is required by the Docker runtime. Multiple app replicas would need organization-controlled shared storage. HTTPS can be supplied by an organization-managed reverse proxy.

The Docker image packages the application, not the 70B model weights or inference server. Model provisioning, compute capacity and operation remain the deployer's responsibility and have not been tested on organization-owned hardware. Initial builds fetch Node and npm dependencies; the default CSCS endpoint requires external network access. We do not claim air-gapped operation. Vercel/Upstash are public demonstration services, not evidence of Swiss-sovereign hosting or the on-premise target.

5. Verification and reproducibility

On 15 October 2026, make check, all 11 automated tests, and the TypeScript/Vite/server production build passed locally. Tests cover strict action response validation, unique reply IDs, encounter isolation, win/loss logic, trusted jump timing, language prompting, all encounter-to-model mappings, rejection route boundaries, authored clips and actual Three.js vertex deformation, plus shared session progress and rejection of conflicting writes.

GitHub Actions Docker verification passed for main commit 04a18f71 and feature commit 2a0577cc. The workflow builds the Docker image, runs tests/build inside it, starts the container with a dummy inference key, and checks health, static frontend and shift creation. This proves container startup and those serving endpoints; it does not prove live inference inside Docker or a complete interactive shift through root make run.

CI evidence: https://github.com/ratafani/Scary-Gate-Keeper/actions/runs/38511334198

The public production health endpoint reports configured Apertus and Redis session storage. A live smoke check on 15 October used four prompts: an English address question, English turn request, German jump request and German negated jump/address request. All four returned the expected action; wall-clock API times were 3.07, 1.64, 1.95 and 2.02 seconds. Two subsequent gate decisions succeeded across visitors. Responses to German prompts mixed English into the speech, so bilingual understanding worked in this sample but German output consistency needs improvement. Full responses are in docs/LIVE_SMOKE.json. This small, single-session sample is not a statistical model evaluation, repair-rate measurement or claim of Swiss German support.

To reproduce: clone the repository; copy track_2b/.env.example to track_2b/.env; set LLM_API_KEY and, if needed, LLM_NAME/LLM_BASE_URL; run make run from the repository root; open http://localhost:3000. Run make down to stop. Judges do not need Xcode, Blender or asset conversion. For development use Node 22, npm ci, npm test and npm run build inside track_2b.

6. Limitations and responsible scope

Remaining verification gaps include a clean root make run plus live inference playthrough, on-premise inference provisioning, broad English/German model reliability measurements, microphone capability, and mobile-device performance coverage. A JSON repair adds latency and does not guarantee valid output. Network failures surface retryable errors. No aggregate accuracy, cost, or concurrency benchmark is claimed.

Authored fictional data and reused assets are supplied by the project owner. The repository is public and source reuse is disclosed. No real-world admission, identity or safety decision should be based on this prototype. Scenario evaluation should record the selected character and prompt because encounter order is randomized.

Code is Apache-2.0; documentation/report is CC-BY-4.0. See LICENSE and NOTICE for provenance. A demo video of at most two minutes accompanies the submission separately; its URL must be supplied in the submission form. No separate Hugging Face dataset submission is proposed.

References

Track requirements: https://hackapertus.notion.site/track-2-b-own-project
Template: https://github.com/HackApertus/project-template
Inference API: https://docs.cscs.ch/services/inference/api/
Model: https://huggingface.co/swiss-ai/Apertus-v1.5-70B